



PAPER • OPEN ACCESS

Supplementing recurrent neural networks with annealing to solve combinatorial optimization problems

To cite this article: Shoummo Ahsan Khandoker *et al* 2023 *Mach. Learn.: Sci. Technol.* **4** 015026

View the [article online](#) for updates and enhancements.

You may also like

- [THE VERTICAL-CURRENT APPROXIMATION NONLINEAR FORCE-FREE FIELD CODE—DESCRIPTION, PERFORMANCE TESTS, AND MEASUREMENTS OF MAGNETIC ENERGIES DISSIPATED IN SOLAR FLARES](#)

Markus J. Aschwanden

- [Impact of random alloy fluctuations on inter-well transport in InGaN/GaN multi-quantum well systems: an atomistic non-equilibrium Green's function study](#)

Michael O'Donovan, Mathieu Luisier, Eoin P O'Reilly et al.

- [TURBULENCE SPECTRA FROM DOPPLER-BROADENED SPECTRAL LINES: TESTS OF THE VELOCITY CHANNEL ANALYSIS AND VELOCITY COORDINATE SPECTRUM TECHNIQUES](#)

A. Chepurnov and A. Lazarian



PAPER

OPEN ACCESS

RECEIVED
20 September 2022REVISED
26 January 2023ACCEPTED FOR PUBLICATION
2 February 2023PUBLISHED
22 February 2023

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Supplementing recurrent neural networks with annealing to solve combinatorial optimization problems

Shoummo Ahsan Khandoker¹ , Jawaril Munshad Abedin¹ and Mohamed Hibat-Allah^{2,3,*} ¹ Department of Computer Science, BRAC University, Dacca, Bangladesh² Department of Physics and Astronomy, University of Waterloo, Vector Institute for Artificial Intelligence, Waterloo, Canada³ Vector Institute, MaRS Centre, Toronto, Ontario, M5G 1M1, Canada

* Author to whom any correspondence should be addressed.

E-mail: mohamed.hibat.allah@uwaterloo.ca**Keywords:** optimization problems, annealing, recurrent neural networks, RNNs, machine learning, statistical physics

Abstract

Combinatorial optimization problems can be solved by heuristic algorithms such as simulated annealing (SA) which aims to find the optimal solution within a large search space through thermal fluctuations. This algorithm generates new solutions through Markov-chain Monte Carlo techniques which can result in severe limitations, such as slow convergence and a tendency to stay within the same local search space at small temperatures. To overcome these shortcomings, we use the variational classical annealing (VCA) framework that combines autoregressive recurrent neural networks (RNNs) with traditional annealing to sample solutions that are uncorrelated. In this paper, we demonstrate the potential of using VCA as an approach to solving real-world optimization problems. We explore VCA's performance in comparison with SA at solving three popular optimization problems: the maximum cut problem (Max-Cut), the nurse scheduling problem (NSP), and the traveling salesman problem (TSP). For all three problems, we find that VCA outperforms SA on average in the asymptotic limit by one or more orders of magnitude in terms of relative error. Interestingly, we reach large system sizes of up to 256 cities for the TSP. We also conclude that in the best case scenario, VCA can serve as a great alternative when SA fails to find the optimal solution.

1. Introduction

Combinatorial optimization is widely used in many areas of science such as physics, computer science, and biology. It also has a wide range of applications in the industry including without limitation to supply chain, energy, and transportation. Providing an efficient solution to combinatorial optimization problems can boost scientific progress and provide optimal solutions to lots of industry problems. Nevertheless, a large class of these problems is NP-Hard [1]. Thus, devising clever algorithms is needed to cope with the exponential growth in the search space that contains all possible combinatorial solutions. In particular, a heuristic algorithm based on the concept of annealing was devised and is known as simulated annealing (SA) or classical annealing (CA) [2]. The latter is inspired by metallurgy, where slowly reducing the temperature of a metal can result in a more stable configuration with a lower energy [2]. Given the Markovian nature of these algorithms, they can result in getting stuck in a local minima due to the rugged nature of some optimization landscapes.

In this paper, we aim to use tools from machine learning for the purpose of improving traditional implementations of SA. In particular, this work is based on the findings of [3] that uses recurrent neural networks (RNNs) to variationally simulate the classical version of annealing within a framework called variational classical annealing (VCA) [3]. Here, we apply RNNs through VCA to find the optimal solution to real-world optimization problems namely the maximum cut problem (Max-Cut), the nurse scheduling problem (NSP), and the traveling salesman problem (TSP). This work has shown that this variational implementation is capable of outperforming SA on average in the asymptotic limit of slow annealing. We note that the use of RNNs in our work allows us to avoid the expensive use of GPU memory resources, and to

reach system sizes beyond 100 cities for the TSP. This limitation can be found in traditional Transformer models [4].

2. Related work

Similar studies have proposed the use of a parameterized ansatz to approximate the ground state of combinatorial optimization problems. In particular, [5] introduces a technique called classical quantum optimization (CQO) where they use neural-network quantum states (NQS) [6] to find the ground state of optimization problems that have been cast into a cost Hamiltonian (also called the energy function) [5, 7, 8]. The NQS ansatz aims to minimize the cost Hamiltonian by training its parameters using the stochastic reconfiguration (or natural gradient) optimization technique [9]. Our study is based on the findings of [3] that adds the annealing component on top of CQO. This study was further extended to the task of finding the ground state of quantum systems with frustration [10].

Furthermore, there have been studies that leverage the Transformer architecture [11] to parameterize the solution distribution of the TSP [4, 12, 13]. The aforementioned papers encode the TSP cities using a transformer encoder but take different approaches to decoding the next city in the partial (uncomplete) tour. Kool *et al* [12] queries the first and last cities in the partial tour, and a combined representation of all the cities to decode the next city. For training, the authors use reinforcement learning (RL) with a greedy baseline. Deudon *et al* [13] decodes the next city with a query consisting of the last three cities in the partial tour, and the training being done with the Actor-Critic RL [13]. Afterwards, the solution generated by the transformer architecture is further improved using the 2-OPT heuristic. Bresson and Laurent [4] decodes the next city autoregressively, meaning the query is constructed using all the cities in the partial tour. The training is performed in an RL framework with a greedy baseline similar to [12].

Additionally, thermal fluctuations have been supplemented to the process of training binary neural networks within an approach known as the Bayesian learning rule [14]. Thermal fluctuations and entropy have also been used to improve Actor-Critic RL under a framework called Soft Actor-Critic RL [15]. We also see the use of annealing in machine learning being put forward in the training process of variational auto-encoders [16]. Similarly to the principle of annealing, the gradual increase in problem complexity has been explored to solve optimization problems under a framework called curriculum learning [17]. We finally note that a summary of the progress made by the machine learning community for the task of solving optimization problems can be found in [18].

3. VCA

For an energy function (or a Hamiltonian) $H(\mathbf{X})$ that encodes an optimization problem, our aim is to find the ground state $\mathbf{X}^*$ by optimizing a set of a model's parameters λ . The VCA algorithm introduced in [3], aims to produce a probability distribution P_λ that approximates the Boltzmann distribution at zero temperature through annealing and optimization of the parameters λ . The peak of this Boltzmann distribution provides the optimal solutions to $H(\mathbf{X})$. This algorithm is inspired by the use of variational autoregressive networks to solve statistical mechanics systems [19].

Autoregressive models originally aim to use the observation in the previous time step as input to the subsequent time step as it usually done in natural language processing. In the context of our work, we harness the power of autoregressive models for the purposes of solving optimization problems by sequentially sampling solutions to these problems [3]. This sampling scheme allows to obtain perfect samples as opposed to Metropolis sampling which can produce correlated samples, and which can get stuck in local minima if the optimization problem has a rugged optimization landscape [20].

Within the setting of VCA, we use RNNs as our model to be optimized. This scheme is performed through minimizing the variational free energy function

$$F_\lambda(t) = \langle H(\mathbf{X}) \rangle_\lambda - T(t)S(P_\lambda). \quad (1)$$

The gradient descent learning on this cost function is used to update λ at every annealing stage. Here, $\langle H(\mathbf{X}) \rangle_\lambda$ is the average Hamiltonian over the RNN distribution P_λ , $T(t)$ denotes the temperature at time t , and $S(P_\lambda)$ is the Shannon entropy. The latter is formulated as

$$S(P_\lambda) = - \sum_i P_\lambda(\mathbf{X}_i) \log(P_\lambda(\mathbf{X}_i)), \quad (2)$$

where the sum iterates over all the states $\{\mathbf{X}_i\}$ in the search space. Computing the true entropy by considering all the states in the exact state space is intractable in general. However, one can approximate the entropy by taking a statistical average over the autoregressive samples obtained from the distribution P_λ as

shown in appendix A. The VCA framework is based on temperature annealing. The latter was found to be helpful in avoiding mode collapse [19]3 and in an adiabatic evolution to the lowest energy [3]. In our annealing scheme, we use a linear annealing schedule characterized by $T(t) = T_0(1 - t)$ where time $t \in [0, 1]$ and T_0 is the initial temperature.

VCA initially performs a series of warm-up steps for the purpose of bringing a randomly initialized RNN probability distribution to the minimum of the free energy $F_\lambda(0)$ by performing N_{warmup} gradient descent steps. Throughout this initial equilibrating process, the temperature is kept constant at $T(0) = T_0$. Once the warm-up steps are completed, the annealing process begins. Any time the temperature is cooled from $T(t')$ to $T(t'')$, the shape of the free energy landscape of the system changes depending on the magnitude of $\Delta t = t'' - t'$. To equilibrate the RNN to a minimum of the new free energy $F_\lambda(t'')$, we perform N_{train} gradient descent steps. This combination of annealing and re-equilibration is performed until $t = 1$ where $T(1) = 0$, marking the end of the annealing process. At this point, the probability distribution P_λ is expected to be maximized around states with an energy equal or close to the ground state energy $H(\mathbf{X}^*)$.

In our implementation, we parameterize time as $t = i/N_{\text{annealing}}$, where $N_{\text{annealing}}$ is a user-defined hyperparameter denoting the number of annealing steps which effectively determines the speed of the annealing process, such that i iterates over $0, \dots, N_{\text{annealing}}$.

4. Optimization problems

4.1. The NSP

We now focus our attention on the optimization problems we tackle using the VCA algorithm. The first one is NSP which belongs to the class of scheduling problems in the field of operations research, and it is known to be NP-hard [21]. NSP aims to assign nurses to specific shifts in a hospital under a set of imposed constraints. The imposed constraints make it hard to find satisfactory solutions to the problem. In this work, we apply VCA to the formulation of NSP introduced in [22].

In this formulation, we have the following constraints: the hard nurse constraint, the hard shift constraint, and the soft nurse constraint. The hardness determines the importance of respecting the constraint. The hard nurse constraint requires that no nurse works for two consecutive shifts as they require sufficient rest after a shift. The hard shift constraint emphasizes the need to deploy enough nurses to handle a given shift. Finally, the soft nurse constraint favors solutions with an even distribution of nurses assigned to shifts. In our paper, the terms “shift” and “day” are synonymous.

To better understand NSP, we take a look at what defines such a problem. In a hospital, we have N individual nurses given by $n \in \{1, \dots, N\}$ and D working days given by $d \in \{1, \dots, D\}$. A solution to the NSP problem could be represented by a matrix

$$\mathbf{X}_M = \begin{bmatrix} x_{1,1} & \dots & x_{1,D} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,D} \end{bmatrix},$$

where $\mathbf{X}_M \in \{0, 1\}^{N \times D}$. If nurse n has been assigned to day d then $x_{n,d} = 1$, otherwise $x_{n,d} = 0$. We use the flattened vector representation of $\mathbf{X}_M$ to represent our solution denoted by $\mathbf{X} \in \{0, 1\}^{ND}$. With index manipulation, the matrix elements of $\mathbf{X}_M$ map to the elements in $\mathbf{X} = [x_{m(1,1)}, \dots, x_{m(N,D)}]$ using $m(n, d) = D(n - 1) + d$.

The three constraints are formulated individually into separate quadratic penalty functions [22]. The hard nurse constraint, which is quantified by

$$H_1(\mathbf{X}) = \sum_{n=1}^N \sum_{d=1}^D \alpha x_{m(n,d)} x_{m(n,d+1)}, \quad (3)$$

penalizes a solution that has instances of a nurse n working two days in a row. Every time this violation occurs a penalty of α is added.

The hard shift constraint has been formulated using the workforce required on a particular day $W(d)$. The constraint Hamiltonian

$$H_2(\mathbf{X}) = \sum_{d=1}^D \left(\sum_{n=1}^N x_{m(n,d)} - W(d) \right)^2, \quad (4)$$

aims to equalize the accumulated contribution of the nurses assigned to a working day and the actual amount of workforce required on that day. A penalty is incurred when there are not enough nurses assigned to a day, or conversely, when there is a surplus of nurses.

Lastly, we have the soft nurse constraint which promotes equal distribution of all nurses across the working days. This constraint is given as follows:

$$H_3(\mathbf{X}) = \sum_{n=1}^N \left(\sum_{d=1}^D x_{m(n,d)} - F(n) \right)^2. \quad (5)$$

The soft constraint term $H_3(\mathbf{X})$ has been formulated using $F(n)$ that gives us the number of days a nurse n wishes to work. If equal distribution of nurses is a desirable outcome, then setting at least $F(n) = \lfloor D/N \rfloor$ for all nurses is required as this ensures a fair workload when D is a multiple of N .

We can sum the three penalty terms shown in equations (3)–(5) to get the resultant energy function

$$H(\mathbf{X}) = H_1(\mathbf{X}) + \lambda H_2(\mathbf{X}) + \gamma H_3(\mathbf{X}), \quad (6)$$

where λ and γ are real coefficients that scale the penalty terms $H_2(\mathbf{X})$ and $H_3(\mathbf{X})$ respectively. $H(\mathbf{X})$ is minimized when, for a given NSP configuration, the least number of constraints are broken. If all the constraints are obeyed then the optimal energy $H(\mathbf{X}^*) = 0$. However, it should be noted that for certain combinations of N and D , it is not always possible for the ground state $\mathbf{X}^*$ to respect all the constraints, resulting in $H(\mathbf{X}^*) > 0$.

Interestingly, given the quadratic form of $H(\mathbf{X})$ the binary nature of $\mathbf{X}$, $H(\mathbf{X})$ can be represented as a quadratic unconstrained binary optimization (QUBO) model [22, 23] such that

$$H(\mathbf{X}) = \sum_{i=1}^{ND} \sum_{j=i}^{ND} q_{ij} x_i x_j + c,$$

where coefficient $q_{i,j}$ and constant c are derived from equation (6). This expression can be made more compact as

$$H(\mathbf{X}) = \mathbf{X}^T \mathbf{Q} \mathbf{X} + c$$

such that $\mathbf{Q} \in \mathbb{R}^{ND \times ND}$ is a square matrix with matrix elements given by q_{ij} .

4.2. The maximum cut problem (Max-Cut)

The second problem we tackle with VCA is the Max-Cut problem that has been known to be at least NP-hard [24]. The Max-Cut problem is defined as follows: given an undirected graph $G(V, E)$, we make a cut along the edges of G to get two complementary sets of vertices such that the number of edges between the two sets is maximized. In other words, if $E_{\text{cut}} \subset E$ is the set of edges bridging the two complementary sets of vertices, we wish to maximize its size $|E_{\text{cut}}|$. In the context of our study, we work with unweighted graphs.

To model the partition across the graph, we can set a value of either 1 or 0 to the vertices to label the partitions where they belong. Therefore, any solution to a graph of N vertices is given by $\mathbf{X} = (x_1, \dots, x_N)$ where $x_i \in \{0, 1\}$. This mapping allows us to use an energy function $H(\mathbf{X})$ that computes the negative of the sum of the number of edges belonging in E_{cut} . This step is done by summing the following expression over all the edges in set E :

$$H(\mathbf{X}) = - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j). \quad (7)$$

Taking the negative converts Max-Cut into a minimization problem. An edge connecting x_i and x_j exists in E_{cut} when $x_i \neq x_j$. The latter provides a contribution of -1 to $H(\mathbf{X})$. For simplicity, we note that the expression of $H(\mathbf{X})$ is equivalent to the following Kronecker Delta expression

$$H(\mathbf{X}) = - \sum_{(i,j) \in E} 1 - \delta_{x_i x_j}.$$

Note that $H(\mathbf{X})$ in equation (7) is quadratic due to the binary nature of variables x_i . Therefore, $H(\mathbf{X})$ can be cast into a QUBO form as follows

$$H(\mathbf{X}) = \mathbf{X}^T \mathbf{Q} \mathbf{X}, \quad (8)$$

where $\mathbf{Q} \in \mathbb{R}^{N \times N}$ is the QUBO square matrix with matrix elements derived from equation (7).

4.3. The TSP

TSP is another famous combinatorial optimization problem that we attempt to solve with VCA. The aim of TSP is to visit a set of cities exactly once and return to the starting city in the tour with a specific order that minimizes the total cost of travel. In our study, the cost is the total distance traveled. We are particularly interested in the 2D Euclidean TSP which is known to be NP-complete [25].

Given N cities on a 2D Cartesian plane, a solution tour looks as $\mathbf{X} = (\pi(1), \dots, \pi(N))$ where π is a permutation of N cities. Each city $\pi(i)$ has a tuple of x and y coordinates $(x_{\pi(i)}, y_{\pi(i)})$. An important property of a valid solution that should be noted is that every city on the tour should be unique. We also remark that the permutation order of cities is translation invariant. The latter property is exploited in the construction of our parameterized model. The energy function of the 2D Euclidean TSP is given by the sum of the Euclidean distances between consecutive cities, including the trip from the last city back to the first as follows

$$H(\mathbf{X}) = \sum_{i=1}^N \sqrt{(x_{\pi(i)} - x_{\pi((i+1)\%N)})^2 + (y_{\pi(i)} - y_{\pi((i+1)\%N)})^2}, \quad (9)$$

where $\%$ denotes the modulo operator.

5. Implementation

As we mentioned in section 3, our VCA implementation uses RNNs as a parameterized probability distribution. With RNNs, a solution $\mathbf{X}$ is built by the sequential realization of its elements x_i . In other words, solutions are sampled according to the probability chain rule

$$P(\mathbf{X}) = P(x_1)P(x_2|x_1) \dots P(x_N|x_{N-1}, \dots, x_2, x_1), \quad (10)$$

where the product of conditional probabilities $P(x_i|x_{<i})$ provides the joint probability of the solution $\mathbf{X}$. Here the nature of x_i depends on the optimization problem. For NSP and Max-Cut, there are N binary variables $x_i \in \{0, 1\}$, whereas for TSP, we have variables $x_i \in \{1, \dots, N\}$ satisfying the condition $x_i \neq x_{i'}$ for $i \neq i'$.

The VCA architecture can be visualized as an N -sized chain of RNN cells as illustrated in figure 1(a). We use the term ‘site’ to refer to any particular position (among N) in this chain. First of all, we look at the case where all RNN cells share the same set of parameters λ [26]. We first start with the description of a Vanilla RNN cell [27]. Here, our formulation of the hidden state $\mathbf{h}_n$ and the conditional probability vector $\mathbf{P}_n$ at site n are given by

$$\mathbf{h}_n = \text{ELU}(W\mathbf{h}_{n-1} + U\mathbf{x}_{n-1} + \mathbf{b}), \quad (11)$$

$$\mathbf{P}_n = \text{Softmax}(V\mathbf{h}_n + \mathbf{c}). \quad (12)$$

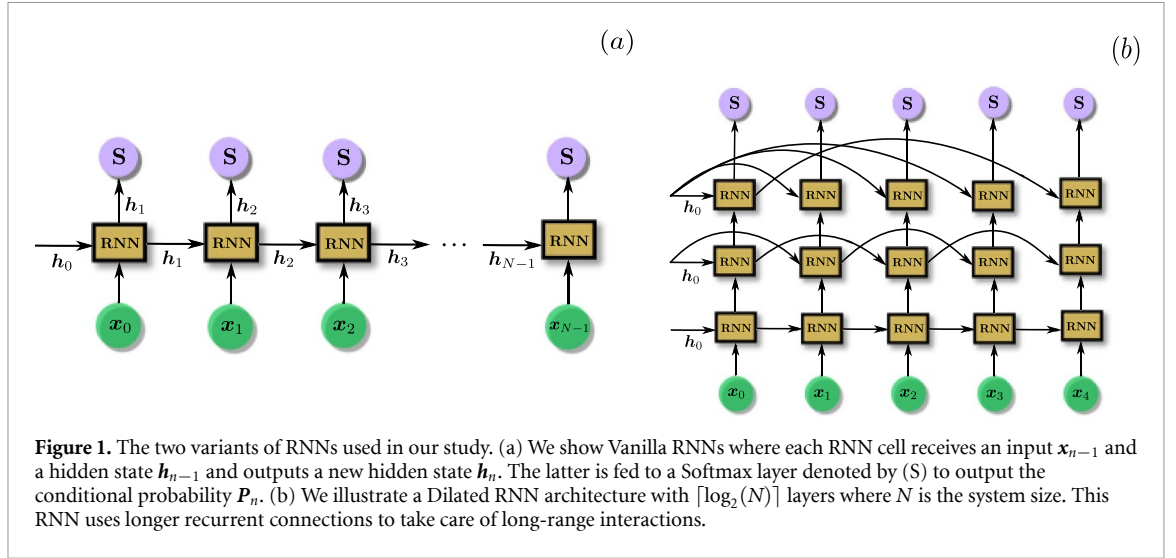
In the above equations, weights W , U , V , and bias vectors $\mathbf{b}$, $\mathbf{c}$ jointly parameterize the RNN cells. In equation (11), $\mathbf{x}_{n-1}$ is the one-hot encoded vector of the variable x_{n-1} sampled by the previous RNN cell. $\mathbf{h}_{n-1}$ is the hidden state from the previous RNN cell. The size of $\mathbf{h}_n$ is a hyperparameter called the number of memory (or hidden) units denoted as d_h . The latter controls the expressivity of the Vanilla RNN cell. Finally, our choice of non-linear activation function is ELU—exponential linear unit [28]. In equation (12), the Softmax function, given by $\text{Softmax}(\sigma_i) = e^{\sigma_i} / \sum_{j=1}^n e^{\sigma_j}$, computes the categorical distribution from which we sample x_n . In this case, the conditional probability of x_n , denoted as $P_\lambda(x_n|x_{<n})$, can be computed as

$$P_\lambda(x_n|x_{<n}) = \mathbf{P}_n \cdot \mathbf{x}_n, \quad (13)$$

where $\cdot$ denotes the dot product operation. The set of N conditional probabilities $\{P_\lambda(x_1), P_\lambda(x_2|x_1), \dots, P_\lambda(x_N|x_{<N})\}$ is used to compute $P_\lambda(\mathbf{X})$ in equation (10).

It should be noted that random interactions between the variables x_i are best captured using RNNs with site-dependent parameters, i.e. each RNN cell has its own set of parameters dedicated to that particular site. This observation is motivated from [3] in which disordered systems, such as Sherrington–Kirkpatrick [29] and Edwards–Anderson [30] spin-glass models, are optimized using site-dependent RNNs. In this case, we have weights $\{W_n\}_{n=1}^N$, $\{U_n\}_{n=1}^N$, $\{V_n\}_{n=1}^N$ and biases $\{\mathbf{b}_n\}_{n=1}^N$, $\{\mathbf{c}_n\}_{n=1}^N$ that are used in a Vanilla RNN architecture as follows:

$$\mathbf{h}_n = \text{ELU}(W_n\mathbf{h}_{n-1} + U_n\mathbf{x}_{n-1} + \mathbf{b}_n) \quad (14)$$



$$P_n = \text{Softmax}(V_n h_n + c_n). \quad (15)$$

We now focus our attention on another RNN architecture to tackle optimization problems with long-range interactions called the Dilated RNN [31]. This architecture uses recurrent skip connections that propagate information from sites that are far from each other as shown in figure 1(b). From the illustration, we also see that this architecture has a stack of Dilated layers that dictate the length of these skip connections given by 2^{l-1} where l is the layer number such that $1 \leq l \leq L$. For a system of size N , we choose $L = \lceil \log_2(N) \rceil$ number of layers by taking inspiration from tensor networks [3, 32]. Here, we index the RNN weights and biases according to the layer l and site n where the RNN cell is located. Thus, the RNN formulation becomes

$$h_n^{[l]} = \text{ELU}(W_n^{[l]} h_{\max(0, n-2^{l-1})}^{[l]} + U_n^{[l]} h_n^{[l-1]} + b_n^{[l]}), \quad (16)$$

$$P_n = \text{Softmax}(V_n h_n^{[L]} + c_n), \quad (17)$$

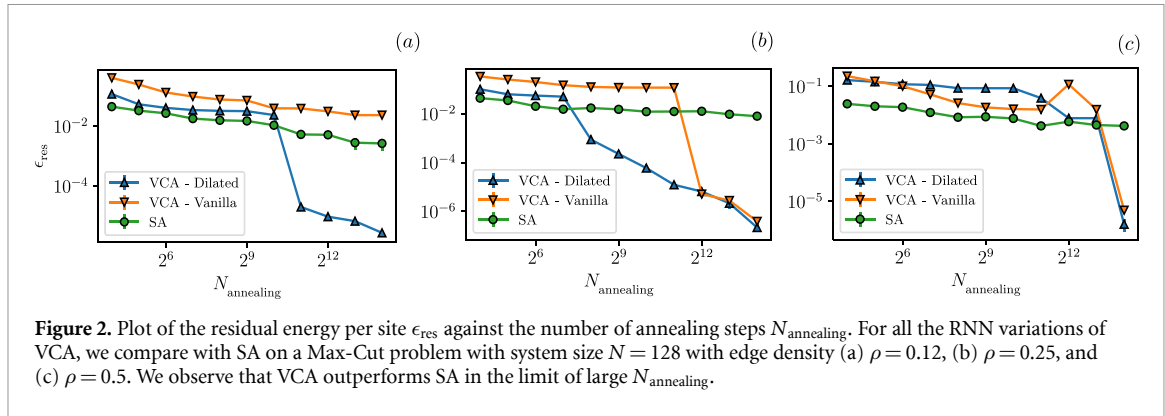
such that $h_n^{[0]} = x_{n-1}$ and $h_0^{[l]} = 0$ for $1 \leq n \leq N$ and $1 \leq l \leq L$.

We note that for the Max-Cut and NSP, we use Vanilla and Dilated RNN with site-dependent parameters to reflect the randomness in the coupling between the discrete variables. Furthermore, given the binary nature of the discrete variables for NSP and Max-Cut, the output probability vector P_n has size 2. For TSP, which enjoys the translation invariance property, we use parameters sharing for the Dilated RNN, i.e. $W_n^{[l]} = W^{[l]}$, $U_n^{[l]} = U^{[l]}$, $V_n = V$, $b_n^{[l]} = b^{[l]}$, $c_n = c$. Additionally the output probability P_n has size N , such that N is the number of cities.

To enforce the constraint of not visiting a city twice in TSP, we apply masking during the process of autoregressive sampling as clarified in appendix B. This trick allows us to avoid the use of an energy penalty to enforce this constraint [33]. The masking also reduces the combinatorial search space. We remark that we conducted experiments where we replace the vanilla RNN cell with LSTM and GRU cells respectively. Compared to the models with vanilla RNN, the GRU and LSTM models do not generally improve on our results. We also note that the use of RNN architectures in our study allows us to go beyond the system sizes that can be reached by Transformer architectures [4, 12]. In particular, we demonstrate that we can go beyond $N > 100$ cities. This advantage is a result of the cheaper computational cost of Dilated RNNs as opposed to traditional Transformer architectures.

6. Experiments and discussion

We now focus our attention on our experiments. Here we compare VCA and SA on the Max-Cut, NSP and TSP optimization problems. For all the problems, we measure the average performance of the two algorithms using the residual energy per site given by $\epsilon_{\text{res}} \equiv (\bar{H}(\mathbf{X}) - H(\mathbf{X}^*)) / N$. This measure quantifies the average error made on the estimation of the ground state energy $H(\mathbf{X}^*)$. Here, N denotes the system size of the problem [34]. For VCA, $\bar{H}(\mathbf{X})$ is estimated by taking an average over the N_{samples} autoregressive samples generated from the RNNs after training. For SA, $\bar{H}(\mathbf{X})$ is approximated by taking the mean over different SA runs. An important observation to highlight is the ability of RNNs to sample a relatively large number of



solutions at the end of annealing compared to SA, which does not offer this possibility. We also consider the best case scenario by computing the minimal residual energy per site defined as $\epsilon_{\text{res}}^{\min} \equiv (H_{\min} - H(\mathbf{X}^*)) / N$ where $H_{\min}$ is the lowest energy obtained by either SA or VCA across the different samples. To make the connection with the literature, we highlight that $\epsilon_{\text{res}}^{\min}$ is directly proportional to the optimality gap defined as $g \equiv (H_{\min} / H(\mathbf{X}^*)) - 1$ [12]. More details about the VCA hyperparameters and the SA implementation can be found in appendix A.

To compare VCA and SA, ϵ_{res} is plotted against the number of annealing steps $N_{\text{annealing}}$ (see section 3). In terms of hardware, we use an Nvidia Titan XP GPU for VCA and a CPU for SA.

6.1. Max-Cut

We use unweighted Max-Cut instances of $N = 128$ vertices and edge densities $\rho = 0.12, 0.25, 0.5$ generated by the Rudy graph generator by G Rinaldi [35]. These graphs were originally generated to benchmark the CQO technique [5]. The choice of increasing densities is used to investigate the effects of adding more complexity to the Max-Cut problem while keeping the number of vertices constant. We approximate the ground state of these graphs using the ConicBundle package by C Helmsberg from the Biq Mac Solver server [36].

We use the Vanilla RNN (see equation (14)) and Dilated RNN (see equation (16)) with site-dependent parameters to run VCA on this problem. As illustrated in figure 2, the general trend we see for both VCA and SA is that ϵ_{res} decreases with the number of annealing steps. VCA with Dilated RNNs (VCA-Dilated) outperforms SA on average starting from annealing steps $2^{11}, 2^8, 2^{14}$ respectively for the densities $0.12, 0.25, 0.5$. Although SA has better energies on average in the fast regime, VCA provides a better convergence at larger number of annealing steps. This observation suggests that VCA requires a threshold number of annealing steps before it consistently converges to the ground state.

If we consider the best solutions obtained by SA and VCA (see table 1), we observe that for all three densities, SA finds the ground state at $N_{\text{annealing}} = 2^4$. For both VCA variants, it is required to have a longer annealing time to find the ground states. The latter means that SA can find the optimal solution of our Max-Cut instances in fewer number of annealing steps compared to VCA.

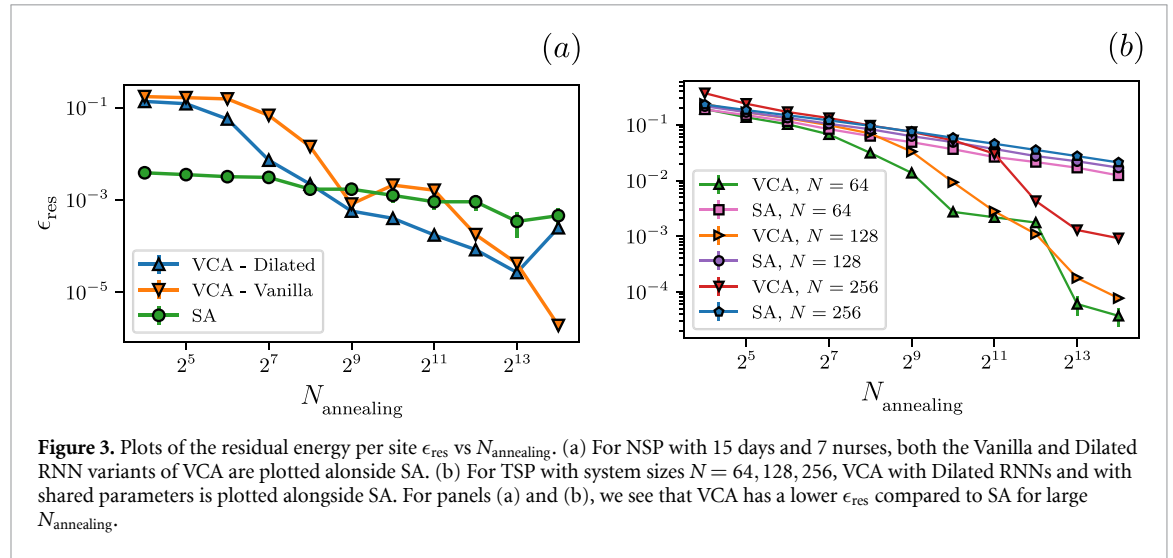
6.2. NSP

For the NSP problem, we use a configuration of 15 days and 7 nurses resulting in a total system size of $15 \times 7 = 105$. Taking inspiration from [22], we use the following NSP parameters described in section 4.1: $W(d) = 1, F(n) = \lfloor D/N \rfloor$, $\alpha = 3.5, \lambda = 1.3, \gamma = 0.3$. Based on the chosen setup, the ground state energy is given as $H(\mathbf{X}^*) = \gamma$. The proof is provided in appendix C.

Similar to the Max-Cut problem, we use Vanilla and Dilated RNNs with site-dependent parameters to tackle this problem. The results are presented in figure 3(a). Again, we see that increasing the number of annealing steps allows VCA to reduce the average error ϵ_{res} . Both VCA-Dilated and VCA-Vanilla reach lower error margins compared to SA at large $N_{\text{annealing}}$. Between annealing steps 2^9 and 2^{13} , VCA-Dilated maintains the lowest average error among all the models. For the slowest annealing schedule, $N_{\text{annealing}} = 2^{14}$, VCA-Vanilla reaches the overall lowest average error, whereas VCA-Dilated sees a sudden increase in ϵ_{res} . A possible reason for this sudden increase can be related to the choice of our hyperparameters, which can be further tuned. We also note that for a small number of annealing steps, SA outperforms VCA on the average case.

Table 1. A summary table of the best performances of VCA and SA on NSP, Max-Cut and TSP. Here we define the minimal residual energy per site $\epsilon_{\text{res}}^{\text{min}} \equiv (H_{\text{min}} - H(X^*)) / N$ where H_{min} is the lowest energy obtained by either SA or VCA across the different samples. ‘D’ stands for the Dilated RNN results and ‘V’ stands for the Vanilla RNN results. Values in bold font correspond to the lowest $N_{\text{annealing}}^*$ to find the exact or the lowest approximation to the ground state after comparing SA and VCA. Bold values also highlight the lowest $\epsilon_{\text{res}}^{\text{min}}$ as well as the lowest estimated time to find the exact or the best solution.

	Parameter	$\epsilon_{\text{res}}^{\text{min}}$	SA $N_{\text{annealing}}^*$	Time	$\epsilon_{\text{res}}^{\text{min}}$	VCA $N_{\text{annealing}}^*$	Time
NSP	15D, 7N	0.0	2^4	3s	D: 0.0 V: 0.0	2^6 2^7	4 min 5 s 1 min 25 s
Max-Cut	$\rho = 0.12$	0.0	2^4	6s	D: 0.0 V: 0.0	2^{11} 2^{13}	45 min 32 s 1 hr 23 min
	$\rho = 0.25$	0.0	2^4	6s	D: 0.0 V: 0.0	2^8 2^{12}	9 min 14 s 42 min 14 s
	$\rho = 0.50$	0.0	2^4	6s	D: 0.0 V: 0.0	2^{14} 2^{14}	5 hrs 35 min 2 hrs 43 min
TSP	$N = 64$	3.13×10^{-3}	2^{14}	1 min 30 s	D: 0.0	2^{13}	49 min 15 s
	$N = 128$	9.24×10^{-3}	2^{14}	6 min 19 s	D: 3.84×10^{-5}	2^{13}	1 hr 24 min
	$N = 256$	1.75×10^{-2}	2^{14}	25 min 42 s	D: 8.60×10^{-4}	2^{14}	6 hrs 51 min



Lastly, by considering the best solutions, SA finds the ground state at $N_{\text{annealing}} = 2^4$ whereas VCA lands at the optimal solution around $N_{\text{annealing}} = 2^6$ as illustrated in table 1. The latter means that SA, similarly to Max-Cut, needs fewer $N_{\text{annealing}}$ steps compared to VCA to find optimal solutions in the best case.

6.3. TSP

For the TSP, we use three instances with varying system sizes $N = 64, 128, 256$. Each component of the coordinates (x, y) of the cities has been uniformly sampled from the range $(0, 1)$. The approximation of the ground state tours of these TSP configurations are provided by the Concorde algorithm [37] through the NEOS Server [38].

Unlike the previous problems, we use VCA with Dilated RNNs that have shared parameters owing to the translation invariance property of TSP. The focus on Dilated RNNs for this problem is motivated by the presence of long-range interactions in TSP, as well as by the overall advantage of Dilated RNNs over Vanilla RNNs for Max-Cut and NSP. The results from our experiments are presented in figure 3(b). For $N = 64, 128$, VCA reaches a residual energy per site $\epsilon_{\text{res}} < 10^{-4}$ whereas for $N = 256$, we reach an average error $\epsilon_{\text{res}} \sim 10^{-3}$. We also observe that SA reaches $\epsilon_{\text{res}} \sim 10^{-2}$ in the slow annealing regime. Overall, in the range of medium to slow annealing, VCA demonstrates a better average performance compared to SA.

By considering the best case scenario, VCA finds the best tours for all three problem sizes. It also finds the exact ground state for $N = 64$ as illustrated in table 1. In contrast, SA finds tours with a higher cost and gets stuck in local minima. Interestingly, this result is different from what we observed for the Max-Cut and the NSP. Thus, we conclude that if a user is interested in finding the best solutions to a particular optimization problem, VCA is a valuable alternative when SA fails to find an optimal solution.

7. Conclusion

In this work, we supplement RNNs with the principle of annealing under a framework called VCA to solve real-world optimization problems. In our experiments, we find that VCA outperforms traditional SA on average in the limit of slow annealing. The latter suggests that a variational simulation of CA is a great alternative for solving optimization problems. When considering the best case scenario, we conclude that VCA is best to use when SA fails to provide a good solution. The failure of SA is likely to occur for models with a glassy landscape, that has a large number of local minima, such as in spin-glass models [39–41]. We also highlight that we reach large system sizes up to 256 cities for the TSP, by virtue of the cheaper cost of RNNs compared to ordinary Transformers [42].

In our results, we see that even though VCA outperforms SA for all three problems on average in the limit of slow annealing, VCA takes more time per iteration compared to SA for the same number of annealing steps (see table 1). This limitation could be mitigated by the use of meta-learning and transfer learning techniques to predict the solutions to optimization problems that are similar to each other within a shorter amount of time [43]. Furthermore, with more hardware advancements, VCA is expected to show speed-up in the running time. We have also seen instabilities that occurred during the training of RNNs and we mitigated some of them by careful choice of the hyperparameters such as tuning the learning rate and varying the number of training samples. Overall, there is significant space for exploration in terms of hyperparameter tuning, hardware efficiency, and choice of architectures.

From a broader perspective, we believe there are various directions of research that could be undertaken in the future. It would be interesting to benchmark VCA against other Monte Carlo optimization algorithms such as parallel tempering [44]. It would be also interesting to extend our work for optimization problems with continuous variables [45] using models that can handle continuous variables [46, 47]. Additionally, one could also incorporate graph autoregressive networks in the VCA scheme to take the graph structure of some optimization problems into consideration [48, 49]. There is also flexibility in tuning the temperature cooling schedules to potentially improve the performance of VCA [50]. We believe that the combination of deep learning techniques and the principle of annealing is an interesting direction of research that could potentially improve our current solutions to optimization problems.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: <https://github.com/RNN-VCA-CO/RNN-VCA-CO> [51].

Acknowledgments

We are grateful to Juan Carrasquilla for providing valuable insights on the manuscript. We also thank Amine Mohamed Aboussalah, Estelle Inack, Jack Raymond, Rochisha Agarwal for valuable discussions. We would like to acknowledge Wasif Ahmed for making the connection between the authors of this paper. We are also grateful for Vector Institute compute resources that made our numerical simulations possible. M H acknowledges support from Mitacs through the Mitacs Accelerate program.

Appendix A. Hyperparameters and training

The VCA hyperparameters are detailed in table A1. Our SA implementation is based on Metropolis moves. At the initial temperatures T_0 , we perform N_{warmup} warmup steps where each step corresponds to a sweep of N Metropolis moves, where N is the number of variables. After this step, we alternate between annealing steps (by decreasing temperature) and equilibrium steps N_{eq} . The latter is similar in spirit to N_{train} in VCA. The hyperparameters of SA (see table A2) are chosen to be consistent with VCA hyperparameters.

To train our RNN architectures, we minimize the variational free energy function $F_{\lambda}(t) = \langle H(\mathbf{X}) \rangle_{\lambda} - T(t)S(P_{\lambda})$. Here we approximate the true variational free energy at any given temperature by taking the average over N_{samples} solutions sampled from the RNN distribution P_{λ} . This approximation is given by

$$F_{\lambda}(t) \approx \frac{1}{N_{\text{samples}}} \sum_{i=1}^{N_{\text{samples}}} (H(\mathbf{X}_i) + T(t) \log(P_{\lambda}(\mathbf{X}_i))). \quad (\text{A1})$$

Furthermore, the gradients of the variational free energy $\partial_{\lambda} F_{\lambda}(t)$ have a similar expression as shown in [3]. We also optimize the relevant parameters λ with the Adam optimizer [52].

Table A1. A table of the hyperparameters used for the VCA experiments in our study.

Hyperparameter	Max-Cut (figure 2)	NSP (figure 3(a))	TSP (figure 3(b))
Architecture	Dilated and Vanilla RNN	Dilated and Vanilla RNN	Dilated RNN with weight sharing
N_{samples}	50 (5×10^5 after annealing)	50 (5×10^5 after annealing)	50 (10^6 after annealing)
N_{train}	5	5	5
N_{warmup}	1000	1000	2000
$N_{\text{annealing}}$	$[2^4, 2^5, \dots, 2^{14}]$	$[2^4, 2^5, \dots, 2^{14}]$	$[2^4, 2^5, \dots, 2^{14}]$
Learning rate	1×10^{-4}	5×10^{-4}	1×10^{-3}
T_0	2.0	2.0	2.0
RNN hidden units	40	40	40
Seed	111	111	111

Table A2. A table of the hyperparameters used for the SA experiments in this paper. N_{samples} corresponds to the number of SA runs similarly to the batch size in VCA.

Hyperparameter	Max-Cut (figure 2)	NSP (figure 3(a))	TSP (figure 3(b))
Metropolis move	Bit flips	Bit flips	Permutations (2-Opt) [53]
N_{samples}	50	50	50
N_{eq}	5	5	5
N_{warmup}	1000	1000	2000
$N_{\text{annealing}}$	$[2^4, 2^5, \dots, 2^{14}]$	$[2^4, 2^5, \dots, 2^{14}]$	$[2^4, 2^5, \dots, 2^{14}]$
T_0	2.0	2.0	2.0
Seed	111	111	111

Appendix B. Masking in TSP

To avoid revisiting the same cities during the processing of autoregressive sampling in VCA, we take inspiration from [54] where constraints were applied in the context of quantum physics, and from [55] in the context of TSP. Here we apply a masking on the visited cities when computing the conditional probabilities $P_{\theta}(c_i | c_{i-1}, \dots, c_2, c_1)$ at the level of the Softmax layer (see equation (13)), where $c_i \in \{1, 2, \dots, N\}$ corresponds to the city to be visited at step i and θ are the parameters of our model.

The masking at step i is done as follows:

- If cities $(c_1, c_2, \dots, c_{i-1})$ were visited, where $c_j \in \{1, 2, \dots, N\}$, then $P_{\theta}(c_j | c_{i-1}, \dots, c_2, c_1)$ is set to zero for $1 \leq j \leq i-1$.
- We renormalize the conditional probability $P_{\theta}(\cdot | c_{i-1}, \dots, c_2, c_1)$ to 1.

After implementing these steps, our autoregressive model generates permutations π of the cities. We note that this masking trick can be implemented in parallel across the batch size.

Appendix C. NSP ground state energy

Instead of deriving the ground state energy for NSP with days $D = 15$ and nurses $N = 7$, we show near identical calculations for NSP with a relatively simpler configuration of $D = 5, N = 2$. The NSP parameters used in our experiment are shown in table C3. Looking at the ground state (see table C3(a)), we see that the hard nurse constraint is respected since neither nurse n_1 nor nurse n_2 has to work two days in a row. Referring back to the penalty functions from section 4.1, we get

$$H_1(X^*) = 0.$$

We also know that each of the days only requires $W(d) = 1$ amount of workforce. Therefore, the hard shift constraint is also satisfied since there is only one nurse assigned each day:

$$H_2(X^*) = 0.$$

However, the soft nurse constraint is broken exactly once because $5\%2 = 1$ where $\%$ is the modulo operator. This observation leaves exactly one day (here d_5) to which a nurse will be assigned to (in this case n_1) who

Table C3. The NSP parameters used in our experiments along with (a) a ground state solution and (b) a sub-optimal solution for NSP with $D = 5, N = 2$.

						α	λ	γ	$W(d)$	$F(n)$	
						3.5	1.3	0.3	1	$\lfloor D/N \rfloor$	
(a) Ground state solution X^* .						(b) Sub-optimal solution X' .					
						d_1	d_2	d_3	d_4	d_5	
n_1	1	0	1	0	1	n_1	1	0	1	0	0
n_2	0	1	0	1	0	n_2	0	1	0	1	0

has to work an extra day. Therefore

$$H_3(X^*) = 1.$$

Summing the penalty functions with their respective scaling, we obtain

$$H(X^*) = H_1(X^*) + \lambda H_2(X^*) + \gamma H_3(X^*) = \gamma.$$

One can notice that it is energetically favorable to break the soft nurse constraint once than the hard shift constraint as $\gamma < \lambda, \alpha$. Thus, a solution X' that does not have any nurse working on d_5 can not be the ground state (see table C3(b)).

ORCID iDs

Shoummo Ahsan Khandoker  <https://orcid.org/0000-0001-7367-4485>

Mohamed Hibat-Allah  <https://orcid.org/0000-0002-5298-8589>

References

- [1] Lucas A 2014 Ising formulations of many np problems *Front. Phys.* **2**
- [2] Kirkpatrick S, Gelatt C D and Vecchi M P 1983 Optimization by simulated annealing *Science* **220** 671–80
- [3] Hibat-Allah M, Inack E M, Wiersema R, Melko R G and Carrasquilla J 2021 Variational neural annealing *Nat. Mach. Intell.* **3** 952–61
- [4] Bresson X and Laurent T 2021 The transformer network for the traveling salesman problem (arXiv:2103.03012)
- [5] Gomes J, McKiernan K A, Eastman P and Pande V S 2019 Classical quantum optimization with neural network quantum states (arXiv:1910.10675)
- [6] Carleo G and Troyer M 2017 Solving the quantum many-body problem with artificial neural networks *Science* **355** 602–6
- [7] Sinchenko S and Bazhanov D 2019 The deep learning and statistical physics applications to the problems of combinatorial optimization (arXiv:1911.10680)
- [8] Zhao T, Carleo G, Stokes J and Veerapaneni S 2020 Natural evolution strategies and variational Monte Carlo *Mach. Learn.: Sci. Technol.* **2** 02LT01
- [9] Sorella S 1998 Green function Monte Carlo with stochastic reconfiguration *Phys. Rev. Lett.* **80** 4558
- [10] Hibat-Allah M, Melko R G and Carrasquilla J 2021 Supplementing recurrent neural network wave functions with symmetry and annealing to improve accuracy *Machine Learning and the Physical Sciences (NeurIPS 2021)*
- [11] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A N, Kaiser Ł and Polosukhin I 2017 Attention is all you need *Advances in Neural Information Processing Systems* vol 30
- [12] Kool W, Herke V H and Welling M 2018 Attention, learn to solve routing problems! (arXiv:1803.08475)
- [13] Deudon M, Cournot P, Lacoste A, Adulyasak Y and Rousseau L-M 2018 Learning heuristics for the TSP by policy gradient *Int. Conf. on the Integration of Constraint Programming, Artificial Intelligence and Operations Research (Berlin: Springer)* pp 170–81
- [14] Meng X, Bachmann R and Khan M E 2020 Training binary neural networks using the Bayesian learning rule
- [15] Haarnoja T, Zhou A, Abbeel P and Levine S 2018 Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor (arXiv:1801.01290)
- [16] Hao F, Chunyuan Li, Liu X, Gao J, Celikyilmaz A, and Carin L 2019 Cyclical annealing schedule: a simple approach to mitigating KL vanishing (arXiv:1903.10145)
- [17] Lisicki M, Afkanpour A and Taylor G W 2020 Evaluating curriculum learning strategies in neural combinatorial optimization (arXiv:2011.06188)
- [18] Bengio Y, Lodi A and Prouvost A 2021 Machine learning for combinatorial optimization: a methodological tour d'horizon *Eur. J. Oper. Res.* **290** 405–21
- [19] Wu D, Wang L and Zhang P 2019 Solving statistical mechanics using variational autoregressive networks *Phys. Rev. Lett.* **122** 080602
- [20] Inack E M, Morawetz S and Melko R G 2022 Neural annealing and visualization of autoregressive neural networks in the Newman–Moore model *Condens. Matter* **7** 38
- [21] Osogami T and Imai H 2000 Classification of various neighborhood operations for the nurse scheduling problem *Int. Symp. on Algorithms and Computation (Berlin: Springer)* pp 72–83

- [22] Ikeda K, Nakamura Y and Humble T S 2019 Application of quantum annealing to nurse scheduling problem *Sci. Rep.* **9** 1–10
- [23] Glover F, Kochenberger G and Du Y 2019 Quantum bridge analytics I: a tutorial on formulating and using QUBO models *4OR* **17** 335–71
- [24] Karp R M 1972 Reducibility among combinatorial problems *Complexity of Computer Computations* (Berlin: Springer) pp 85–103
- [25] Papadimitriou C H 1977 The Euclidean travelling salesman problem is NP-complete *Theor. Comput. Sci.* **4** 237–44
- [26] Goodfellow I, Bengio Y and Courville A 2016 *Deep Learning* (Cambridge, MA: MIT Press)
- [27] Lipton Z C, Berkowitz J, and Elkan C 2015 A critical review of recurrent neural networks for sequence learning (arXiv:1506.00019)
- [28] Clevert D-Ae, Unterthiner T and Hochreiter S 2015 Fast and accurate deep network learning by exponential linear units (ELUs) (arXiv:1511.07289)
- [29] Sherrington D and Kirkpatrick S 1975 Solvable model of a spin-glass *Phys. Rev. Lett.* **35** 1792
- [30] Edwards S F and Anderson P W 1975 Theory of spin glasses *J. Phys. F: Met. Phys.* **5** 965
- [31] Chang S, Zhang Y, Han W, Yu M, Guo X, Tan W, Cui X, Witbrock M, Hasegawa-Johnson M and Huang T S 2017 Dilated recurrent neural networks (arXiv:1710.02224)
- [32] Vidal G 2008 Class of quantum many-body states that can be efficiently simulated *Phys. Rev. Lett.* **101** 110501
- [33] Vargas-Calderón V, Parra A N, Vinck-Posada H and González F A 2021 Many-qudit representation for the travelling salesman problem optimisation *J. Phys. Soc. Japan* **90** 114002
- [34] Santoro G E, Martoňák R, Tosatti E and Car R 2002 Theory of quantum annealing of an Ising spin glass *Science* **295** 2427–30
- [35] Helmberg C (available at: www-user.tu-chemnitz.de/~helmberg/rudy.tar.gz)
- [36] Biq Mac Solver 2010 (available at: <https://biqmac.aau.at/>)
- [37] Applegate D, Bixby R, Chvátal Všek and Cook W 2001 TSP cuts which do not conform to the template paradigm *Computational Combinatorial Optimization: Optimal or Provably Near-Optimal Solutions* (Berlin: Springer) pp 261–303
- [38] NEOS Server: concorde (available at: <https://neos-server.org/neos/solvers/co:concorde/TSP.html>)
- [39] Hamze F, Raymond J, Pattison C A, Biswas K and Katzgraber H G 2020 Wishart planted ensemble: a tunably rugged pairwise Ising model with a first-order phase transition *Phys. Rev. E* **101** 052102
- [40] Nokura K 1987 An heuristic approach to the structure of local minima of the Sherrington-Kirkpatrick model *J. Phys. A: Math. Gen.* **20** L1203–5
- [41] TayaraniN M-H and Prügel-Bennett A 2014 On the landscape of combinatorial optimization problems *IEEE Trans. Evol. Comput.* **18** 420–34
- [42] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A N, Kaiser L and Polosukhin I 2017 Attention is all you need (arXiv:1706.03762)
- [43] Zhao T, Stokes J, Knitter O, Chen B and Veerapaneni S 2020 Meta variational Monte Carlo (arXiv:2011.10614)
- [44] Earl D J and Deem M W 2005 Parallel tempering: theory, applications and new perspectives *Phys. Chem. Chem. Phys.* **7** 3910–16
- [45] Fan Q, Huang H, Chen Q, Yao L, Yang K and Huang D 2022 A modified self-adaptive marine predators algorithm: framework and engineering applications *Eng. Comput.* **38** 3269–94
- [46] Papamakarios G, Nalisnick E, Rezende D J, Mohamed S and Lakshminarayanan B 2019 Normalizing flows for probabilistic modeling and inference (arXiv:1912.02762)
- [47] Rezende D and Mohamed S 2015 Variational inference with normalizing flows *Proc. 32nd Int. Conf. on Machine Learning (Proc. of Machine Learning Research vol 37)* ed F Bach and D Blei (Lille: PMLR) pp 1530–8
- [48] You J, Ying R, Ren X, Hamilton W L and Leskovec J 2018 Graphrnn: generating realistic graphs with deep auto-regressive models (arXiv:1802.08773)
- [49] Schuetz M J A, Kyle Brubaker J and Katzgraber H G 2022 Combinatorial optimization with physics-inspired graph neural networks *Nat. Mach. Intell.* **4** 367–77
- [50] Mills K, Ronagh P and Tamblyn I 2020 Finding the ground state of spin Hamiltonians with reinforcement learning *Nat. Mach. Intell.* **2** 509–17
- [51] Github repository 2022 (available at: <https://github.com/RNN-VCA-CO/RNN-VCA-CO>)
- [52] Kingma D P and Jimmy B 2014 Adam: a method for stochastic optimization (arXiv:1412.6980)
- [53] Martoňák R, Santoro G E and Tosatti E 2004 Quantum annealing of the traveling-salesman problem *Phys. Rev. E* **70** 057701
- [54] Hibat-Allah M, Ganahl M, Hayward L E, Melko R G and Carrasquilla J 2020 Recurrent neural network wave functions *Phys. Rev. Res.* **2** 023358
- [55] Bello I, Pham H, Le Q V, Norouzi M and Bengio S 2016 Neural combinatorial optimization with reinforcement learning (arXiv:1611.09940)